DSA 2050 – Data Science Methodology
Week 6 Guided Practical Lab: Regression and Baseline Comparison

Duration: 2.5–3 hours | Tools: Python, Jupyter Notebook, pandas, scikit-learn, matplotlib | Submission: GitHub

This guided lab walks through regression modelling from baseline construction to error analysis. Students will compare simple baselines, Linear Regression, Ridge Regression, and a shallow Decision Tree using MAE, RMSE, and R^2. The emphasis is on sound methodology, leakage-safe preprocessing, training-only cross-validation, residual analysis, subgroup error checks, and interpretation.

Learning Outcomes

- Explain why every regression model should be compared with a baseline.
- Separate features (X) from a continuous target (y).
- Create a defensible train/test split.
- Build a preprocessing pipeline for numeric and categorical variables.
- Fit mean and median dummy regressors, Linear Regression, Ridge Regression, and a shallow Decision Tree.
- Interpret MAE, RMSE, and R^2.
- Use cross-validation on training data only.
- Interpret residuals, subgroup errors, and signs of overfitting.

Part 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
```

```
RANDOM_STATE = 42
```

Why set RANDOM_STATE? Some operations involve randomness. A fixed random state makes the split and modelling results reproducible.

Part 2 – Load the Dataset

```
df = pd.read_csv("DSA2050_Week6_Sales_Data.csv")
df.head()

print("Shape:", df.shape)
print()
df.info()
```

Student check: state the number of rows and columns, identify numeric and categorical variables, explain what one row represents, and identify the prediction target.

Part 3 – Basic Data Audit

```
print("Missing values:")
display(df.isna().sum())

print("Duplicate rows:", df.duplicated().sum())

display(df.describe(include="all"))

df["Sales_KES"].describe()

plt.figure(figsize=(8,4))
plt.hist(df["Sales_KES"], bins=25)
plt.title("Distribution of Sales")
plt.xlabel("Sales (KES)")
plt.ylabel("Frequency")
plt.show()
```

Interpretation: write 2–3 sentences on the target range, shape, and any unusual values. Do not remove extreme values automatically; first consider whether they may be genuine business cases.

Part 4 – Define the Analytical Task

- Business question: Can available business characteristics be used to estimate sales?
- Analytical task: supervised regression.
- Target: Sales_KES.
- Features: AdvertisingSpend_KES, StoreVisits, DiscountPercent, Region, CustomerSegment, ProductCategory.

Part 5 – Separate Features and Target

```
X = df.drop(columns="Sales_KES")
y = df["Sales_KES"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

Part 6 – Train/Test Split

Reserve 20% of the data as an untouched final test set. Use the training set for model development and cross-validation. Do not repeatedly use the test set to choose the model.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=RANDOM_STATE
)

print("Training rows:", len(X_train))
print("Test rows:", len(X_test))
```

Part 7 – Identify Feature Types

```
numeric_features = [
    "AdvertisingSpend_KES",
    "StoreVisits",
    "DiscountPercent"
]

categorical_features = [
    "Region",
    "CustomerSegment",
    "ProductCategory"
]
```

Part 8 – Leakage-Safe Preprocessing

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
```

```
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

Why use a pipeline? Learned preprocessing should be fitted only on training data. Keeping preprocessing inside the pipeline helps prevent information leakage during validation.

Part 9 – Build Baselines

A baseline gives a minimum reference level. The mean baseline predicts the training-set mean for every case. The median baseline predicts the training-set median.

```
mean_baseline = DummyRegressor(strategy="mean")
median_baseline = DummyRegressor(strategy="median")

mean_baseline.fit(X_train, y_train)
median_baseline.fit(X_train, y_train)

mean_pred = mean_baseline.predict(X_test)
median_pred = median_baseline.predict(X_test)
```

Part 10 – Evaluation Metrics

- MAE: average absolute prediction error; lower is better.
- RMSE: penalizes large errors more heavily than MAE; lower is better.
- R^2: how much variation in the target is explained by the model; higher is generally better.

```
def evaluate_regression(y_true, y_pred, name):
    return pd.DataFrame({
        "Model": [name],
        "MAE": [mean_absolute_error(y_true, y_pred)],
        "RMSE": [np.sqrt(mean_squared_error(y_true, y_pred))],
        "R2": [r2_score(y_true, y_pred)]
    })

mean_results = evaluate_regression(y_test, mean_pred, "Mean Baseline")
median_results = evaluate_regression(y_test, median_pred, "Median
Baseline")

pd.concat([mean_results, median_results], ignore_index=True)
```

Part 11 – Linear Regression

```
linear_model = Pipeline([
    ("preprocess", preprocessor),
    ("model", LinearRegression())
])
```

```
linear_model.fit(X_train, y_train)
linear_pred = linear_model.predict(X_test)

linear_results = evaluate_regression(
    y_test, linear_pred, "Linear Regression"
)
linear_results
```

Interpretation questions: Is MAE lower than the baseline? Is RMSE lower? Is R^2 higher? Does the model add useful predictive value?

Part 12 – Ridge Regression

```
ridge_model = Pipeline([
    ("preprocess", preprocessor),
    ("model", Ridge(alpha=1.0))
])

ridge_model.fit(X_train, y_train)
ridge_pred = ridge_model.predict(X_test)

ridge_results = evaluate_regression(
    y_test, ridge_pred, "Ridge Regression"
)
ridge_results
```

Ridge is linear regression with regularization. It discourages very large coefficients and can improve stability when predictors are noisy or correlated.

Part 13 – Shallow Decision Tree

```
tree_model = Pipeline([
    ("preprocess", preprocessor),
    ("model", DecisionTreeRegressor(
        max_depth=4,
        min_samples_leaf=8,
        random_state=RANDOM_STATE
    ))
])

tree_model.fit(X_train, y_train)
tree_pred = tree_model.predict(X_test)

tree_results = evaluate_regression(
    y_test, tree_pred, "Decision Tree"
```

```
)
tree_results
```

Part 14 – Compare All Models

```
comparison = pd.concat([
    mean_results,
    median_results,
    linear_results,
    ridge_results,
    tree_results
], ignore_index=True)

comparison.sort_values("MAE")
```

Write a short paragraph: identify the lowest MAE, lowest RMSE, highest R^2, and the model you would choose from this table alone. Explain why one table is still not enough for a final conclusion.

Part 15 – Cross-Validation on Training Data Only

```
scoring = {
    "MAE": "neg_mean_absolute_error",
    "RMSE": "neg_root_mean_squared_error",
    "R2": "r2"
}

models = {
    "Linear Regression": linear_model,
    "Ridge Regression": ridge_model,
    "Decision Tree": tree_model
}

rows = []

for name, model in models.items():
    scores = cross_validate(
        model, X_train, y_train,
        cv=5,
        scoring=scoring
    )

    rows.append({
        "Model": name,
        "CV_MAE": -scores["test_MAE"].mean(),
        "CV_RMSE": -scores["test_RMSE"].mean(),
        "CV_R2": scores["test_R2"].mean()
```

```
    })

cv_results = pd.DataFrame(rows)
cv_results
```

Questions: Which model performs best on average? Are the cross-validation results consistent with the test results? Why is training-data cross-validation safer than repeatedly checking the final test set?

Part 16 – Residual Analysis

Residual = Actual − Predicted. Positive residuals mean the model under-predicted; negative residuals mean the model over-predicted.

```
residuals = y_test - linear_pred

residual_df = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": linear_pred,
    "Residual": residuals.values
})

residual_df.head()
plt.figure(figsize=(8,5))
plt.scatter(linear_pred, residuals)
plt.axhline(0)
plt.title("Linear Regression Residual Plot")
plt.xlabel("Predicted Sales (KES)")
plt.ylabel("Residual: Actual - Predicted")
plt.show()
```

Interpretation: discuss whether residuals are centered around zero, whether a pattern is visible, and whether large residuals exist.

Part 17 – Actual vs Predicted

```
plt.figure(figsize=(8,5))
plt.scatter(y_test, linear_pred)
plt.title("Actual vs Predicted Sales")
plt.xlabel("Actual Sales (KES)")
plt.ylabel("Predicted Sales (KES)")
plt.show()
```

If predictions were perfect, points would lie close to an imaginary diagonal line where Actual = Predicted.

Part 18 – Inspect Largest Errors

```
error_analysis = X_test.copy()
error_analysis["ActualSales"] = y_test
error_analysis["PredictedSales"] = linear_pred
error_analysis["Residual"] = (
    error_analysis["ActualSales"] - error_analysis["PredictedSales"]
)
error_analysis["AbsoluteError"] = error_analysis["Residual"].abs()

error_analysis.sort_values(
    "AbsoluteError", ascending=False
).head(10)
```

Questions: Do the largest errors share characteristics? Are they concentrated in one region, segment, or category? What additional data might explain them?

Part 19 – Error by Customer Segment

```
segment_errors = error_analysis.groupby(
    "CustomerSegment"
).agg(
    Observations=("AbsoluteError", "size"),
    MAE=("AbsoluteError", "mean")
).reset_index()

segment_errors
```

Explain which segment has the highest MAE, whether sample sizes are similar, and why subgroup performance matters even when overall MAE looks acceptable.

Part 20 – Check for Overfitting

```
tree_train_pred = tree_model.predict(X_train)

print(
    "Shallow Tree Training MAE:",
    round(mean_absolute_error(y_train, tree_train_pred), 2)
)

print(
    "Shallow Tree Test MAE:",
    round(mean_absolute_error(y_test, tree_pred), 2)
)
```

Part 21 – Demonstrate an Overly Deep Tree

```
deep_tree = Pipeline([
    ("preprocess", preprocessor),
```

```
    ("model", DecisionTreeRegressor(
        random_state=RANDOM_STATE
    ))
])

deep_tree.fit(X_train, y_train)

deep_train_pred = deep_tree.predict(X_train)
deep_test_pred = deep_tree.predict(X_test)

print(
    "Deep Tree Training MAE:",
    round(mean_absolute_error(y_train, deep_train_pred), 2)
)

print(
    "Deep Tree Test MAE:",
    round(mean_absolute_error(y_test, deep_test_pred), 2)
)
```

Explain why an almost-perfect training result does not necessarily indicate a better model.

Part 22 – Extension Challenge

Choose ONE:

- Try Decision Tree max_depth values 2, 4, 6, and None and compare test MAE.
- Try Ridge alpha values 0.1, 1, 10, and 100.
- Remove AdvertisingSpend_KES or StoreVisits, retrain Linear Regression, and explain the performance change.

Part 23 – Final Recommendation

Write 6–8 sentences answering: Which baseline was strongest? Which candidate model performed best? Did it clearly beat the baseline? What does MAE mean in business terms? What did residual analysis reveal? Did performance differ by customer segment? Is there evidence of overfitting? Would you deploy the model immediately? Why or why not?

Part 24 – Limitations

State at least three limitations. Examples: small dataset, teaching/synthetic data, limited predictors, missing external factors, future data drift, or the fact that prediction does not imply causation.

GitHub Submission

```
DSA2050-Week6-StudentID/
├── DSA2050_Week6_Guided_Regression_Lab.ipynb
├── DSA2050_Week6_Sales_Data.csv
└── README.md
```

README should include student name/ID, lab objective, models compared, best model, selected model MAE, two key findings, and three limitations. Before submitting, restart the notebook kernel and run all cells from top to bottom.

Suggested Marking Scheme – 20 Marks

Component	Marks
Data inspection and task definition	2
Train/test split and preprocessing	3
Baselines	3
Linear/Ridge regression	2
Decision tree	2
Cross-validation and metric interpretation	3
Residual and segment error analysis	3
Extension challenge	1
Final recommendation and limitations	1
Total	20

Key lesson: A model is useful only when it meaningfully improves on a reasonable baseline, is evaluated without leakage, and its errors are understood well enough to support a real decision.